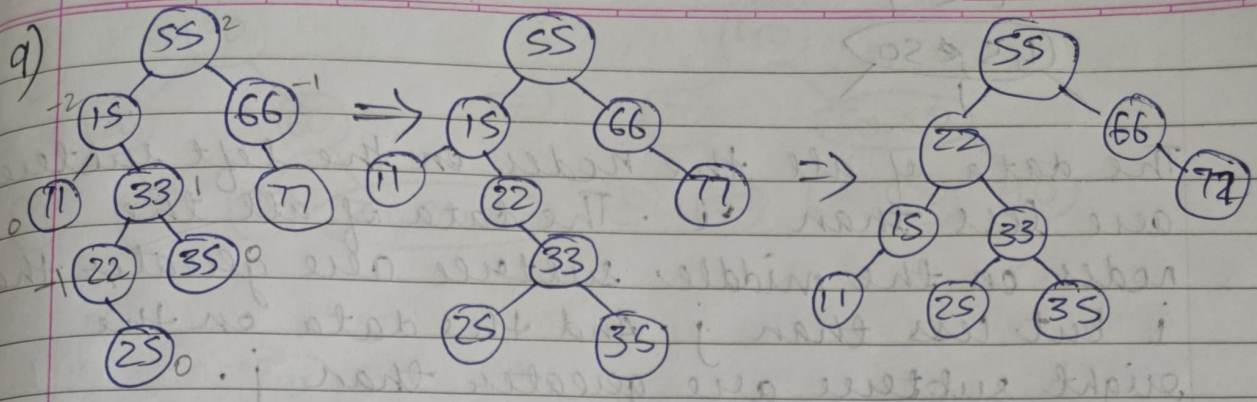


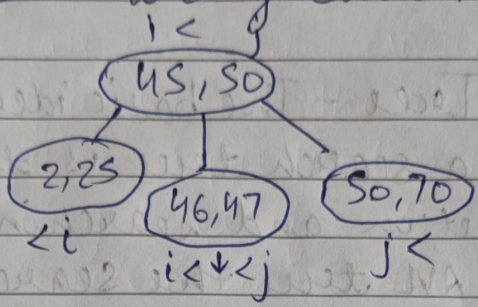


* 2-3 Tree - The basic idea behind maintaining a search tree is to make the insertion deletion and searching operation efficient. In AVL tree the searching operation is efficient however the insertion & deletion involves rotation that makes the operation complicated. To eliminate this complication a data struc. called 2-3 Tree can be used. To build a 2-3 tree there are certain rules that need to be followed -

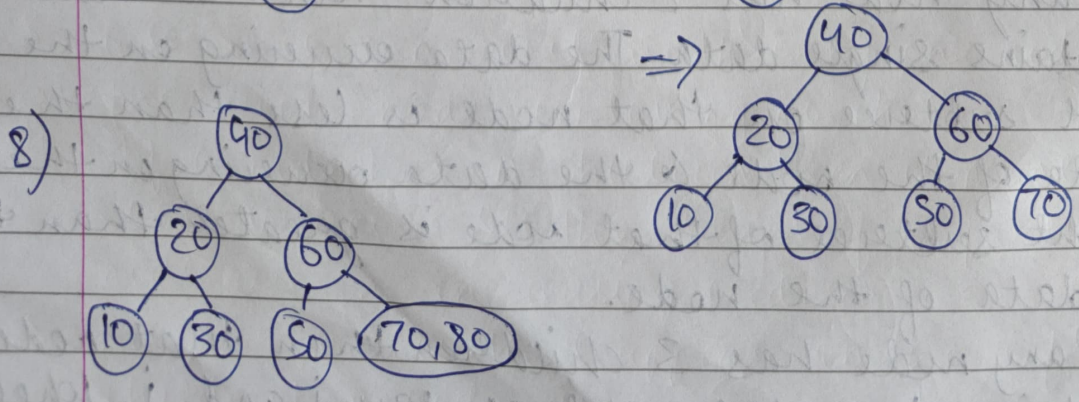
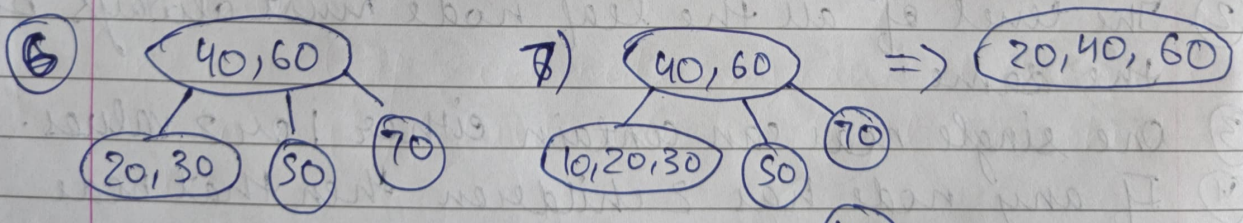
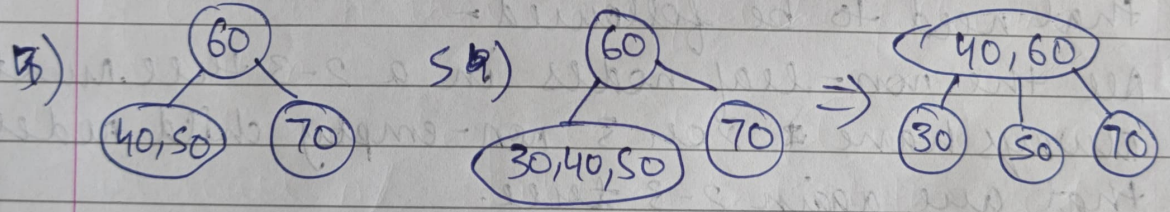
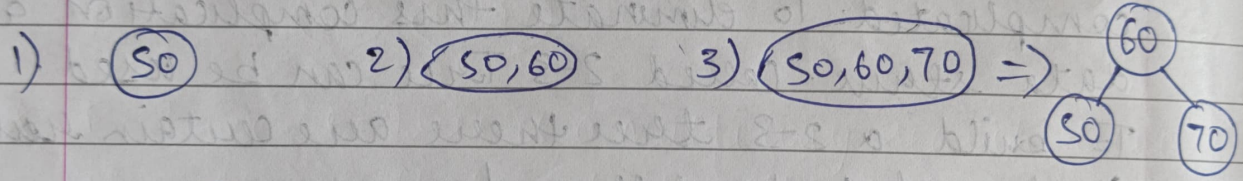
- ① All the non-leaf nodes in a 2-3 tree must always have 2 or 3 non-empty child nodes that are again 2-3 tree.
- ② The level of all the leaf node must always be the same.
- ③ One single node can contain either 1 or 2 values.
- ④ If any node has 2 children then that node contains single data. The data occurring on the left subtree on that node is less than the data of the node & the data occurring on the right subtree of that node is greater than the data of the node.
- ⑤ If any node has 3 children then that node contains two data values, say i and j , where $i < j$.

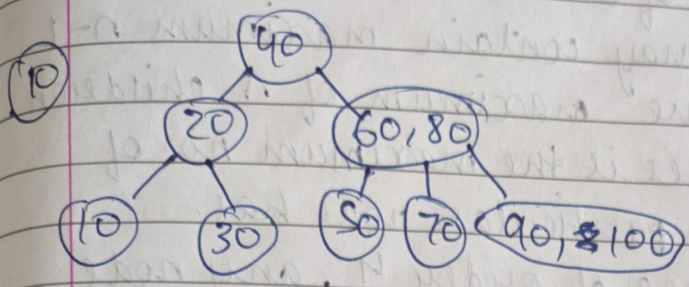
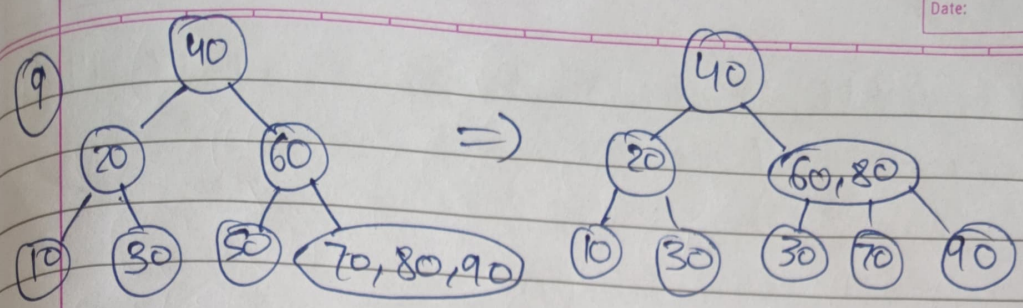
$i < j$
 $(45, 50)$

The data of all the nodes on the left subtree are less than 'i'. The data of all the nodes on the middle subtree are greater than i but less than j. And the data on the right subtree are greater than j.

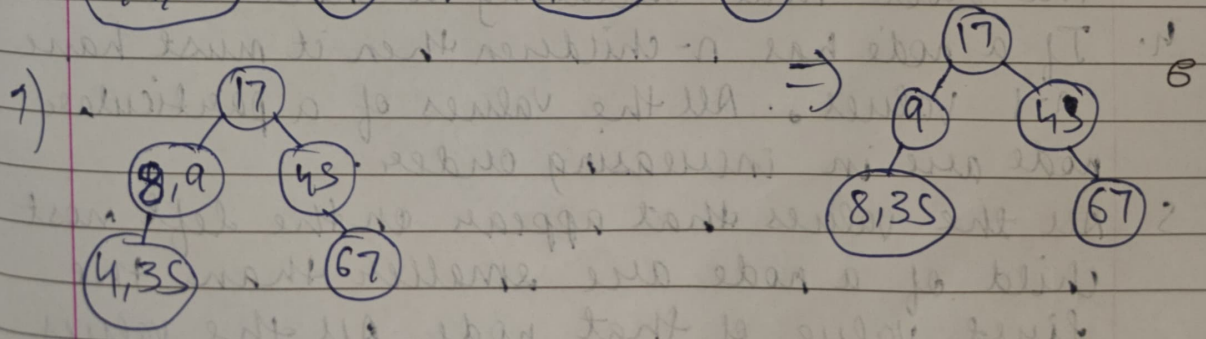
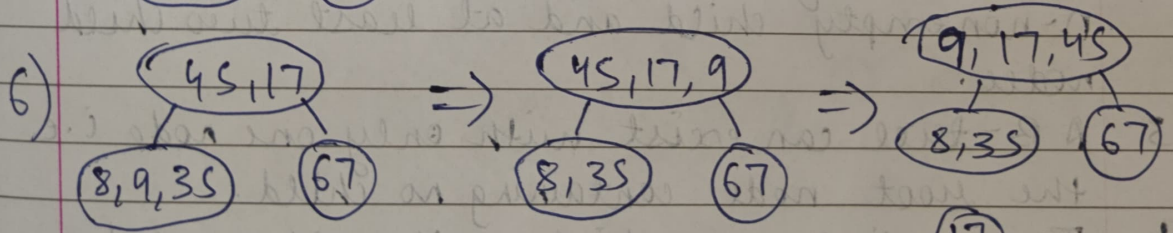
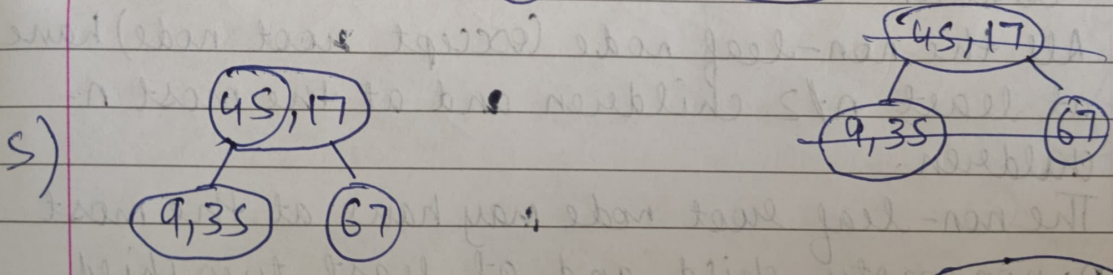
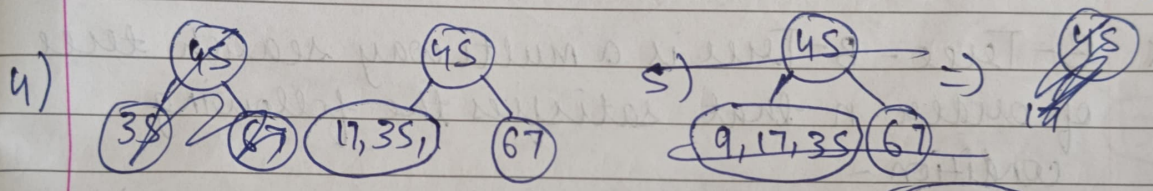
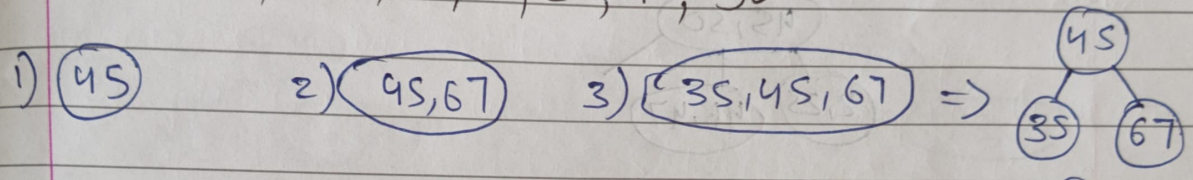


50, 60, 70, 40, 30, 20, 10, 80, 90, 100





45, 67, 35, 17, 9, 8, 4, 50

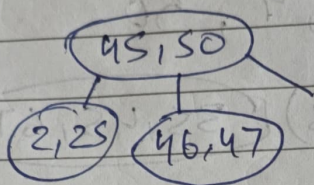


* Multi Way Tree

A multi way tree of order n is a tree in which any node may contain maximum $n-1$ values and can have maximum of n children.

The order of a tree is the maximum no. of child node that a particular node has.

In a multi way tree of order 4, any node can contain maximum 3 values and 4 children.



* B-Tree - B-Tree is a multiway search tree of order n that satisfies the following condition -

1. All the non-leaf node (except root node) have at least $n/2$ children and at the most n children.
2. The non-leaf root node may have at the most n non-empty child and at least two child nodes.
3. A B-tree can exist with only one node i.e. the root node containing no child.
4. If a node has n children then it must have $n-1$ values. All the values of a particular node are in increasing order.
5. All the values that appear on the left most child of a node are smaller than the first value of that node. All the values

that appear on right most child of a node are greater than the last value of that node.

- 6 If x and y are any two i^{th} and $(i+1)^{\text{th}}$ values of a node, where $x < y$, then all the values appearing on $(i+1)^{\text{th}}$ subtree of that node are greater than x and less than y .
- 7 All the leaf nodes should appear on the same level.

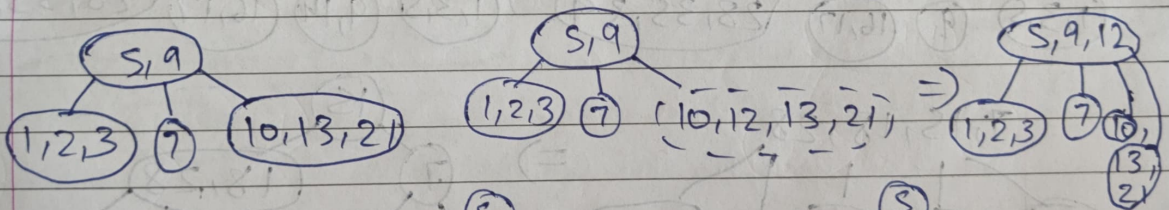
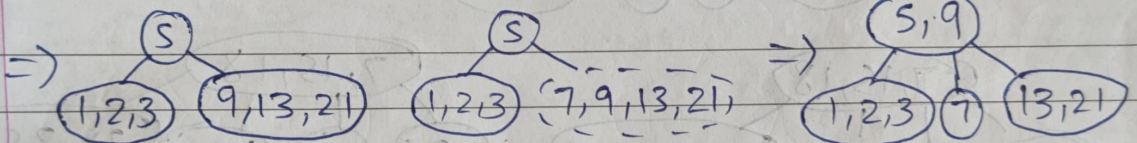
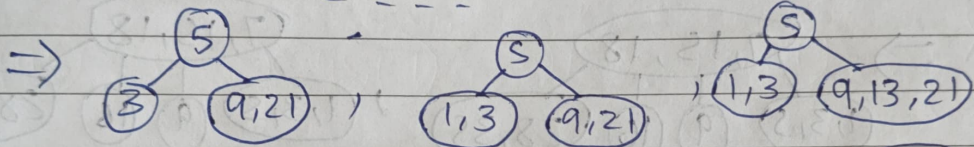
5, 3, 21, 9, 1, 13, 2, 7, 10, 12, 4, 8

(5), (3, 5), (3, 5, 21)

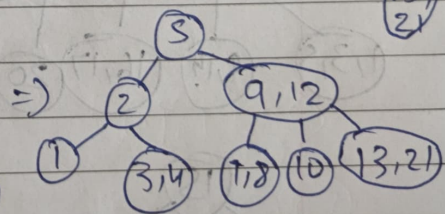
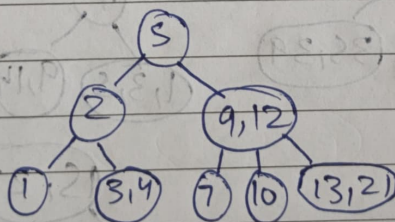
(3, 5, 9, 21)

Order - 4
max - 4
child

max = 4 - 1 = 3
Key

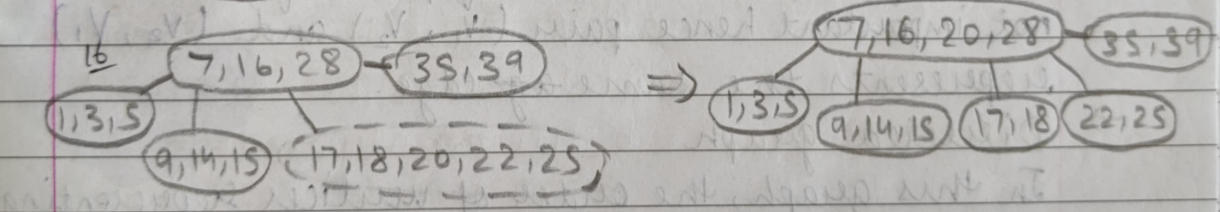
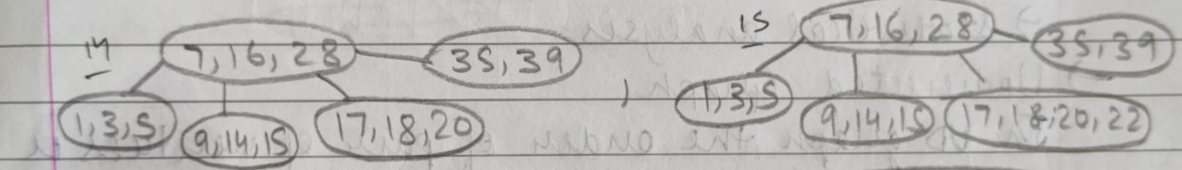
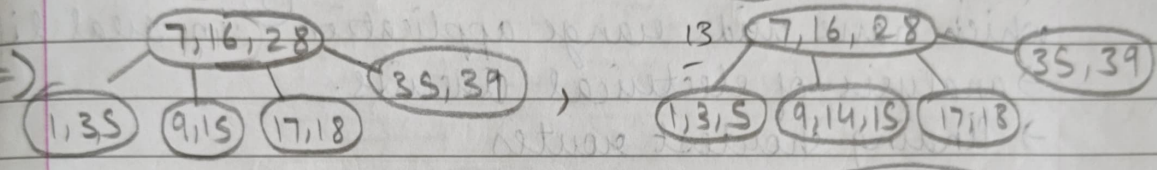
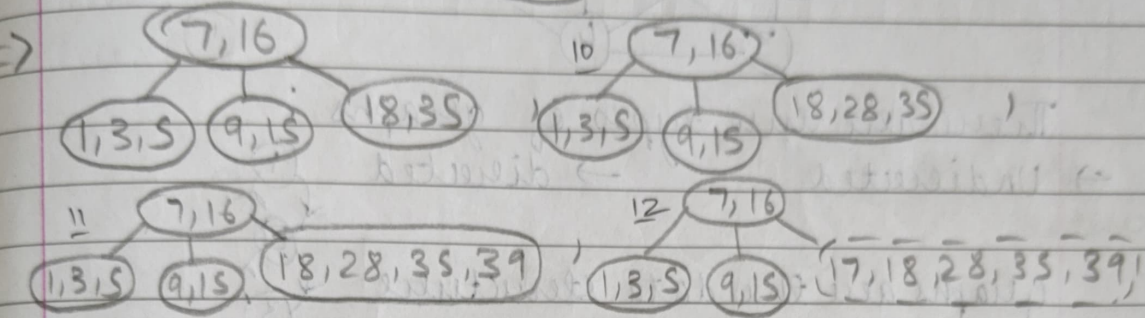
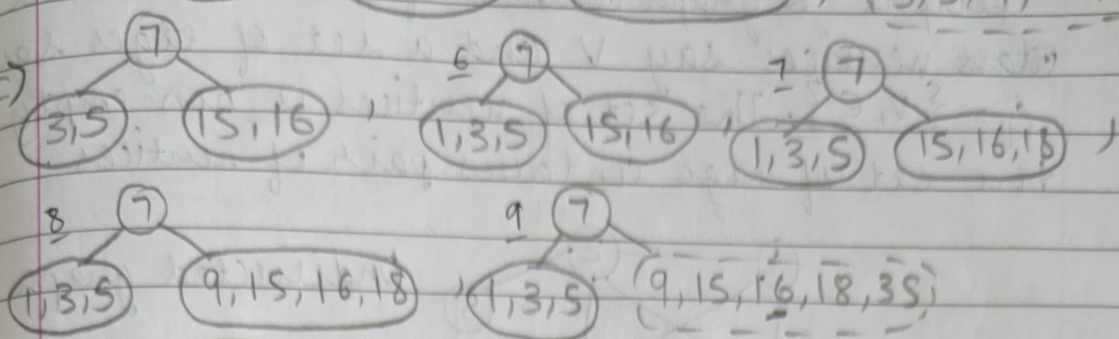


~~5, 9, 12~~



7, 15, 3, 16, 5, 1, 18, 9, 35, 28, 39,
17, 14, 20, 22, 25, 27

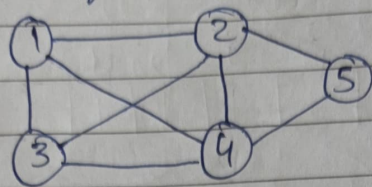
1 (7), 2 (7, 15), 3 (3, 7, 15), 4 (3, 7, 15, 16), 5 (3, 5, 7, 15, 16),



~~$7x^2 + 5x + 3$~~
 ~~$8x^3 + 9x^2 + 7$~~

$8x^3 + 7x^2 + 14x + 10$

Graphs - A graph consists of two sets - a set of ~~edges~~ vertices say V and a set of edges say E . $G = \{V, E\}$. The set of vertices is non-empty & the set of edge contains pair of vertices.



Undirected graph

There are two types of graphs :-

→ Undirected

→ directed



Graphs are the data structure which have wide range application in real life -

→ analysis of electrical circuits.

→ finding shortest routes

→ statistical analysis

→ Undirected Graph

In UD graph, the order of pair of vertices is not important hence pair (V_1, V_2) and (V_2, V_1) represents the same ~~ag~~ edge.

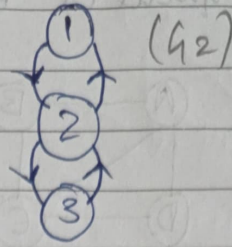
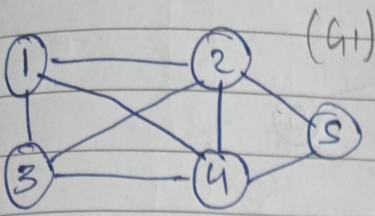
→ Directed graph

In this graph, the order of vertices representing an edge is important. This pair is represented as $\langle V_1, V_2 \rangle$ where V_1 is the tail & V_2 the head of edge. Thus, $\langle V_2, V_1 \rangle$ and $\langle V_1, V_2 \rangle$ represent two different edges.

Adjacent vertices - and incident edges

In an UD graph, if (V_1, V_2) is an edge in the set of edges, then the vertices $\{V_1, V_2\}$ are said to be adjacent & the edge (V_1, V_2) is incident on V_1 and V_2 .

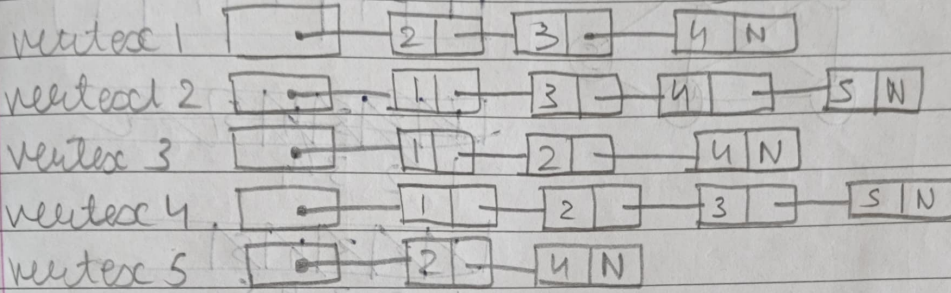
Graph Representation



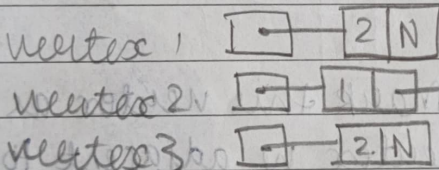
	1	2	3	4	5
1	0	1	1	1	0
2	1	0	1	1	1
3	1	1	0	1	0
4	1	1	1	0	1
5	0	1	0	1	0

	1	2	3
1	0	1	0
2	1	0	1
3	0	1	0

(G1) Adjacency Matrix



(G1)



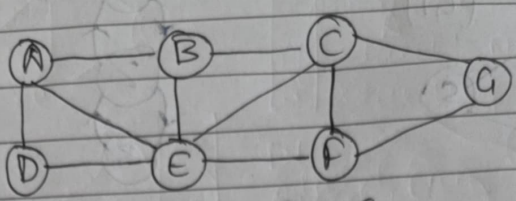
(G2)

Adjacency List

TRAVERSAL IN A GRAPH

- Breadth First Search Algorithm
 - Start traversing the given graph with the given no. of vertex and edges.
 - Insert the starting source vertex (S) to the queue and mark it as visiting visited.
 - Repeat:-
 - Remove the front node K of the queue and examine it.
 - Insert all the adjacent nodes of K (which are not visited so far) to the queue & mark them visited.

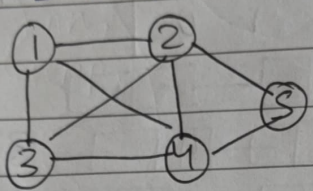
- 4 Until queue is empty.
- 5 Exit



A	B	C	D	E	F	G
1	1	1	1	1	1	1

→ visited nodes
queue

A	B	C	D	E	F	G
A	B	C	D	E	F	G



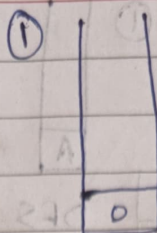
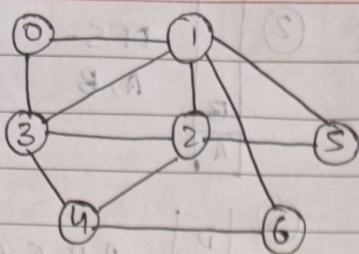
①	②	③	④	⑤
1	1	1	1	1

1	2	3	4	5
1	2	3	4	5

1	2	3	4	5
1	2	3	4	5

→ Depth First Search

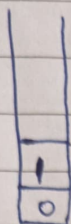
- 1 Visit a vertex, say V. Mark this vertex as visited
- 2 Select an unvisited vertex w adjacent to V.
- 3 Repeat steps a & b till all adj. vertices of w are visited :-
- 4 On reaching vertex whose all adj. vertices have been visited go back to the last vertex visited which has an unvisited vertex adjacent to it & go back to step a.
- 5 Terminate the search when no unvisited vertex can be reached from any of the visited ones.



DFS Seq.

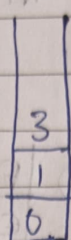
0

②



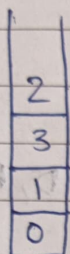
DFS seq
0, 1

③



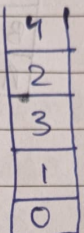
DFS Seq
0, 1, 3

④



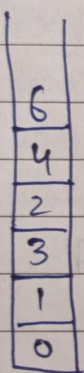
DFS seq
0, 1, 3, 2

⑤



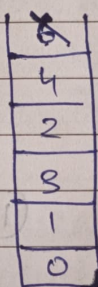
DFS Seq.
0, 1, 3, 2, 4

⑥



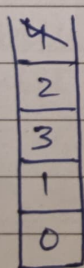
DFS seq.
0, 1, 3, 2, 4, 6

⑦



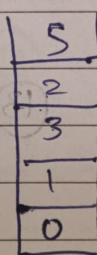
DFS Seq
0, 1, 3, 2, 4, 6

⑧



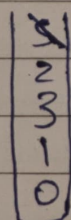
DFS Seq
0, 1, 3, 2, 4, 6

⑨



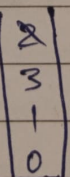
DFS Seq
0, 1, 3, 2, 4, 6, 5

⑩



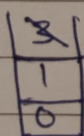
DFS
0, 1, 3, 2, 4, 6, 5

⑪



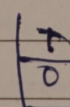
DFS Seq.
0, 1, 3, 2, 4, 6, 5

⑫



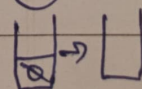
DFS Seq.
0, 1, 3, 2, 4, 6, 5

⑬

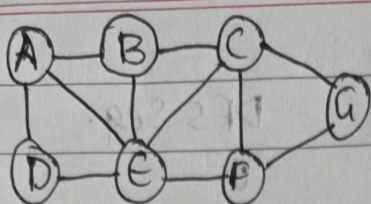


DFS Seq.
0, 1, 3, 2, 4, 6, 5

⑭



0, 1, 3, 2, 4, 6, 5



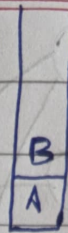
①



DFS

A

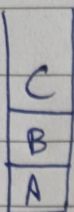
②



DFS-

A, B

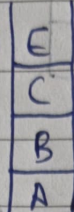
③



DFS-

A, B, C

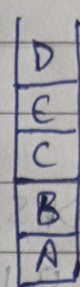
④



DFS

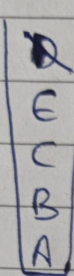
A, B, C, E

⑤



A, B, C, E

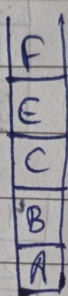
⑥



Trace back

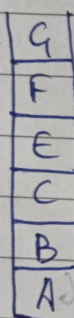
A, B, C, E, D

⑦



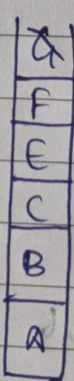
A, B, C, E, D, F

⑧



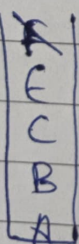
A, B, C, E, D, F, G

⑨



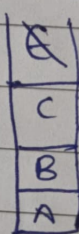
A, B, C, E, D, F,

⑩



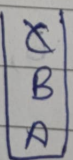
A, B, C, D, F, G

⑪



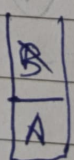
ABCEDFG

⑫



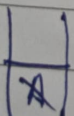
ABCEDFG

⑬

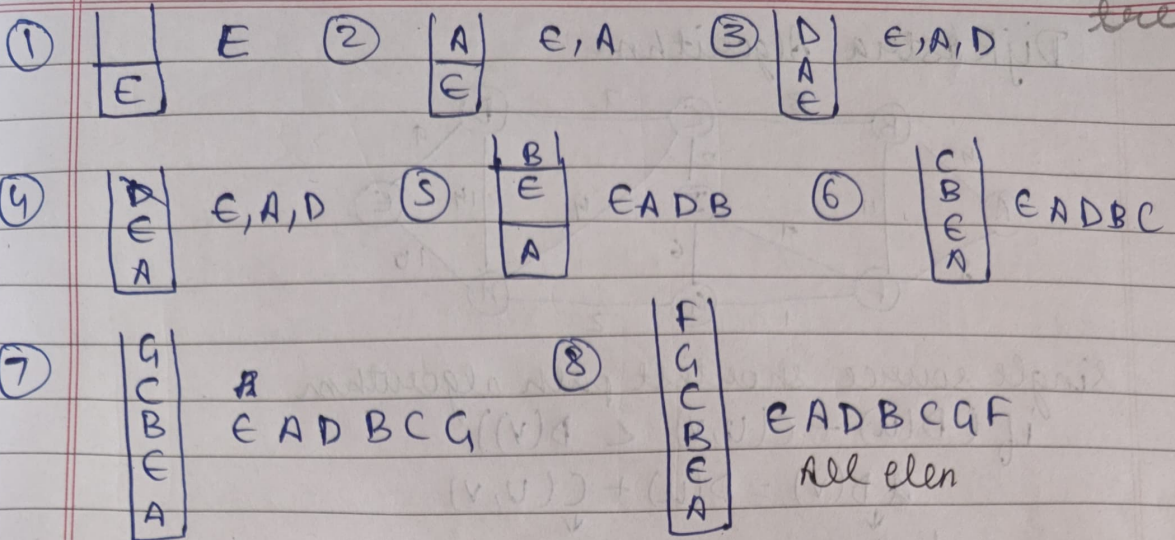


ABCEDFG

⑭

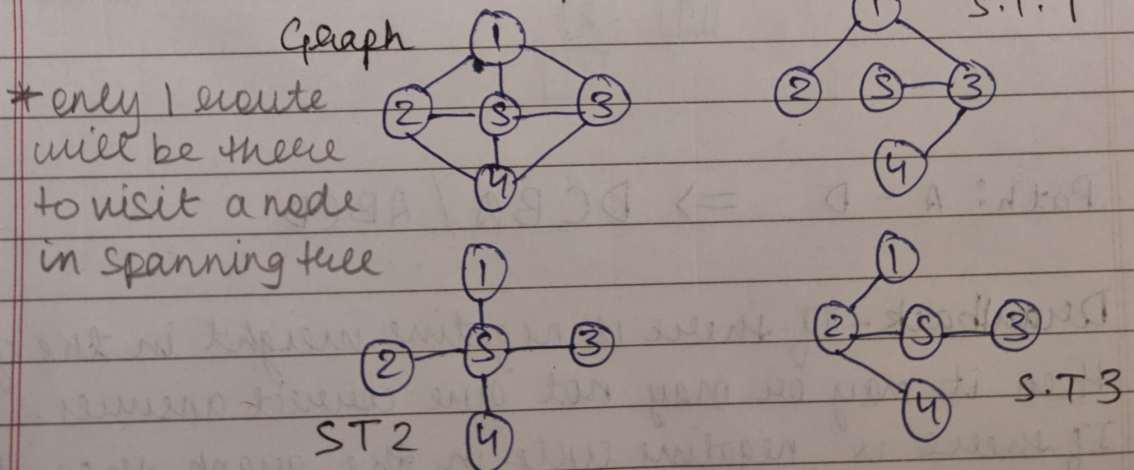


* a graph may have a loop or a child can have multiple parents, unlike a tree

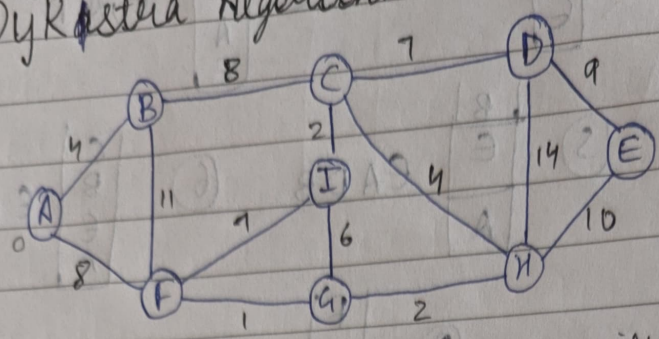


Spanning Tree - A spanning tree of a graph is an undirected tree consisting of only those edges that are necessary to connect all the vertices in the original graph. A spanning tree has a property that for any pair of vertices there exists only one path between them. The spanning tree is useful in

- Analysis of electrical circuit
- Shortest route problem.



Dijkstra Algorithm



Single source shortest path algorithm
 if $(D(u) + C(u, v) < D(v))$
 $\Delta D(v) = D(u) + C(u, v)$
 ↓ ↓
 distance cost

considering 'A' as source node

	A	B	C	D	E	F	G	H	I
A	0	∞	∞	∞	∞	∞	∞	∞	∞
B		4	∞	∞	∞	8	∞	∞	∞
F			12	∞	∞	8	∞	∞	∞
G			12	∞	∞		9	∞	15
H			12	∞	∞			11	15
C			12	25	21				15
I				19	21				11
D				19	21				
E					21				

Path: A - D $\Rightarrow$ DCBA / ABCD

Drawback - If there is negative weight in the graph then it may or may not give correct answer.
 If there is negative cycle in the graph then it will always give wrong answer.

* Searching is very easy direct or linear Searching

Application - DBMS

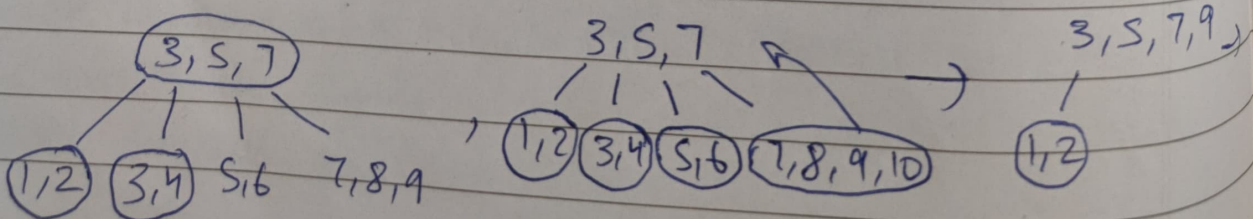
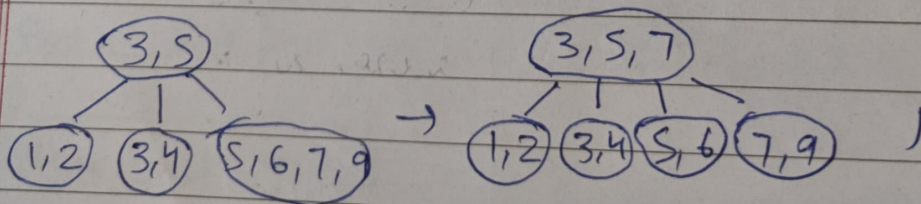
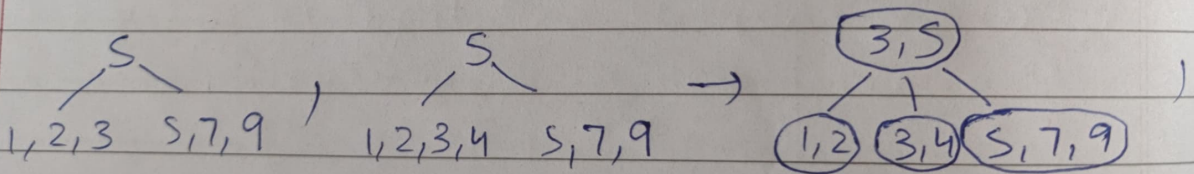
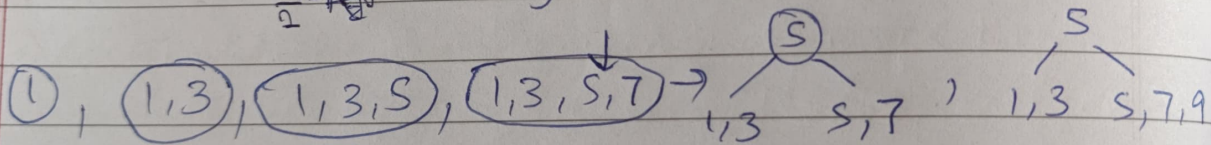
B+ Tree is an extension of B Tree which allows efficient insertion, deletion & search operation. In B+ Tree, data can only be stored on the leaf node while internal node can only store key values. The internal node of B+ Tree are often called index node. The leaf node of a B+ Tree are linked together in form of single linked list to make search query more efficient.

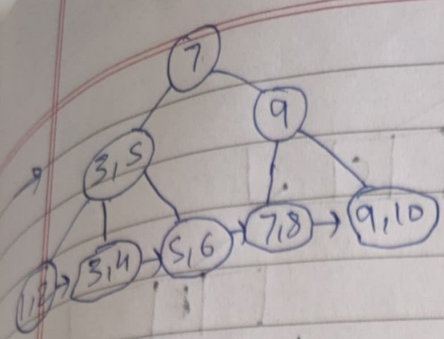
B+ Tree are used to store large amount of data which cannot be stored in main memory. Due to the fact that the size of main memory is always limited. The int. nodes of B+ Tree are stored in main memory whereas leaf nodes are stored in secondary memory.

1, 3, 5, 7, 9, 2, 4, 6, 8, 10 order = 4

$m-1$ max Key = 3 max child = 4

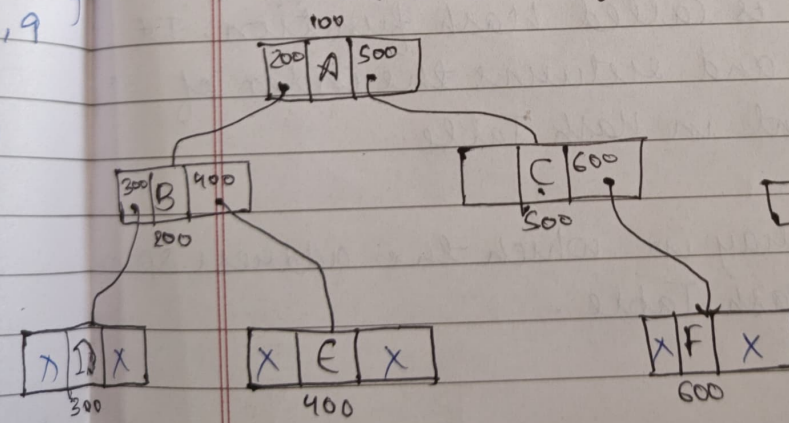
$\frac{m-1}{2}$ min Key = 2 min child = 2 $m/2$



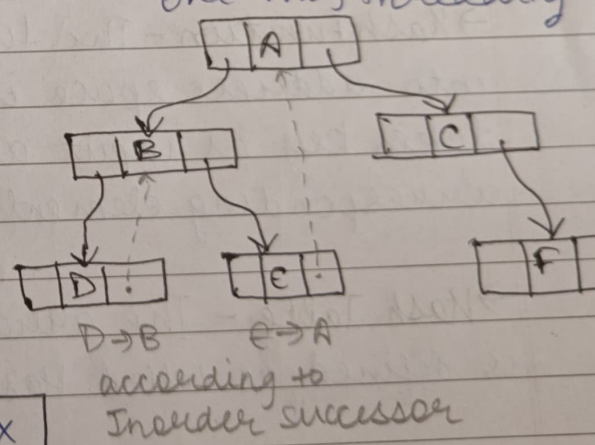


Threaded Binary Tree - is a special kind of binary tree in which the null entries of left and right subtree are replaced by a special pointer called thread. A Thread is a special link that not only avoids the null entries but it also simplifies the traversal on binary tree. The use of stack in non-recursive traversal of a tree requires extra memory & pointer. This problem can be avoided by TBT and makes the tree traversal easier. There are two types of Threaded BT -

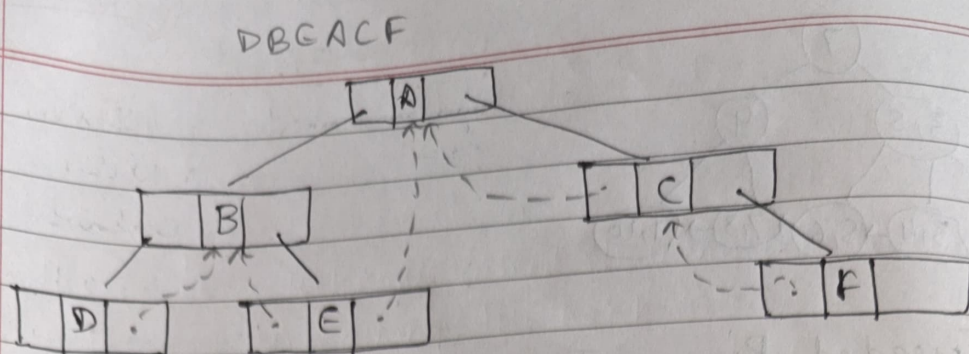
1. One way Threading
2. Two way Threading



one way threading



inorder - DBEACF



Hashing - Hashing is a searching technique that avoids no. of comparisons and goes directly where the required data is present is called hashing. Hashing searches the element in constant time, no matter where the element is located. Hashing is a key to address transformation in which the key map to address in a list.

→ **Key** - Input to the hash function is called a Key. It could be an integer or string.

→ **Hash Function** - The function that maps key space into address space is called Hash function. It takes key as input and returns the index of corresponding element in Hash Table.

→ **Hash Table** - The array in which the address space is defined is called Hash Table.

1) **Division Method** -

$$\text{Hash}(\text{Key}) = \text{Key} \% \text{table size}$$

32, 46, 77, 11

- 1 $\text{Hash}(32) = 32 \% 10 = 2$
- 2 $46 \% 10 = 6$
- 3 $77 \% 10 = 7$
- 4 $11 \% 10 = 1$

00P	0
11	1
32	2
	3
	4
	5
46	6
77	7
	8
	9

within the hash table so that it produce very few collision as possible.
 We must minimize collision

CLASSMATE

Date _____

Page _____

Mid Square

Key	Square
32	1024
46	2116
77	5929
11	121

32	0
46	1
11	2
	3
	4
	5
	6
	7
	8
77	9

Select a middle digit either at 2nd or 3rd place (here 2nd)

* A good hash function should

Folding Method

have the following charac

2246, 1372, 3414

↓	↓	↓
22	13	34
+ 46	+ 72	+ 14
68	85	48

① It maybe formed by using mathematical transformation

② It should be very easy & quick to compute.

2246 will be stored at 68 index
 1372 will be stored at 85 index
 ③ It should produce a relatively random & unique distribution of value

Collision - When hash function maps to the same location in the hash table for two different Keys then this is called collision. This collision occurs when two distinct input in a hash function produce identical output.

32, 46, 77, 42, 37

$$32 \div 10 = 2$$

Collision Resolution Techniques

① Open Hashing

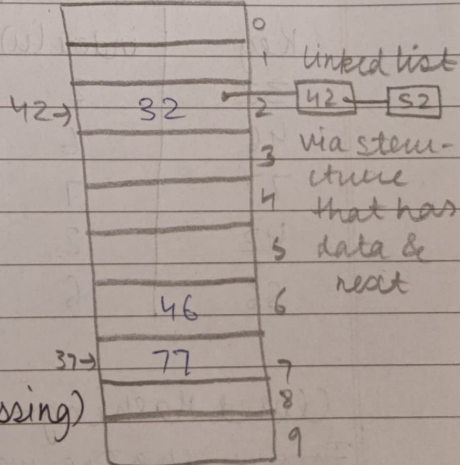
→ chaining

② Closed Hashing (open addressing)

→ Linear Hashing probing

→ Quadratic Hash probing

→ Double Hashing



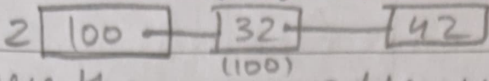
4174
31
4247

classmate

Date _____
Page _____

The nos which

Array of Pointers



here the address of first node will be stored i.e the start of linked list

There is a technique called chaining for dealing with collision. In chaining, the hash array is a collection of pointers where each pointer points to a separate list of elements maintained in the linked list. In this method the hash value represents the address of the linked list that contains all the elements of the same hash value. The header node are kept in the hash array & the actual elements are kept in the linked list. Therefore chaining is implemented by using array where each element of array is a linked list.

② Closed Hashing

→ Linear Probing

32, 46, 77, 42, 56

$$[u + i] \% m$$

$m = \text{Hash Table size} = 10$

$i = 0 \text{ to } m - 1$

Key	index (u)	Probe
32	2	1 (no. of times searched position)
46	6	1
77	7	1
42	2	2
56	6	3

	0
	1
32	2
42	3
	4
	5
46	6
77	7
56	8
	9

$$42 = (2 + 0) \% 10 = 2$$

$$42 = (2 + 1) \% 10 = 3$$

$$56 = (6 + 0) \% 10 = 6$$

$$56 = (6 + 1) \% 10 = 7$$

$$56 = (6 + 2) \% 10 = 8$$

Closed Hashing -

This is the one way to resolve collision is to search for an open or unoccupied place in the hash table where the colliding element can be placed. There are two ways to search for an open place.

compared to linear probing, quadratic probing
 into few elements to search because of squaring. Then

classmate

Date

Page

Linear Probing - This is the simplest way of resolving the collision. In this we compute the hash function. If the hash table already contains the elements then we go for the next location of the hash table. If that location is also filled then we go for the another location next in the hash table using the formula $(u+1) \% m$ until an empty position is found & record is placed at that position.

→ Quadratic Probing -

$$h(K) = 2K + 3$$

Division Method Collision

Key - 3, 4, 7, 5, 8

Resolution technique = Q.P

		Key	Index (u)	Probing	$(u+i^2) \% m$ ($m=0 \leq m-1$)
0	8				
1	4	3	$2 \cdot 3 + 3 = 9 \% 10 = 9$	1	
2		4	$2 \cdot 4 + 3 = 11 \% 10 = 1$	1	
3	5	7	$2 \cdot 7 + 3 = 17 \% 10 = 7$	1	$8 = (9 + 0^2) \% 10 = 9$
4		5	$2 \cdot 5 + 3 = 13 \% 10 = 3$	1	$8 = (9 + 1^2) \% 10$
5		8	$2 \cdot 8 + 3 = 19 \% 10 = 9$	2	$= 0$
6					
7	7				
8					
9	3				

→ Double Hashing $(u + v \cdot i) \% m$
 (Re Hashing)

→ Quadratic Probing - This allows the quick movement of the key within a considerable distance from the initial collision, by generating the random sequence of positions instead of linear sequence. This method examines certain cells away from the original probe point. When the collision occurs the quad. probing applies the following function $(u+i^2) \% m$. One major disadvantage of quad. probing is that it requires time to square the probe no. & it may not

visit all the index of hash table.